DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Searching Text

CMSC 131 Laboratory Activity 3

Prepared by: Rene Andre Bedonia Jocsing

Published: November 12, 2026 **Deadline:** December 09, 2026

This laboratory activity is about arrays and strings. You'll build `rengrep`, a working subset of the `grep` command, in NASM. Correctness is defined by agreement with `grep`, which is already installed on your machine.

Background

Text processing looks easy from a language with a string type. `line.find(pattern)` is one call. Somebody else wrote it.

In assembly there's no string type. There's a region of bytes and a convention about where it ends. Searching for a pattern inside a line means comparing bytes one at a time, at every position, and knowing when to stop. Reading a file into a buffer and treating it as a list of lines means finding the newlines yourself and remembering where each line started.

x86 has instructions built for this job. `cmpsb` compares two bytes and advances two pointers. `scasb` scans for a byte. With the `rep` prefixes they repeat until a condition fails, so a comparison loop that would take five instructions takes one. They're the reason this activity is about arrays and strings together. The string instructions read memory through the index registers, so using them well means understanding addressing.

The convenient part of this activity is the oracle. Your tool has to agree with `grep`, byte for byte. There's no ambiguity about what correct means and no reference implementation to write. The reference is already on your PATH.

Learning Objectives

By the end of this laboratory activity, students should be able to:

- Read a file into a buffer and index it as an array of lines
- Search for a substring using the x86 string comparison instructions
- Fold case without destroying the source data
- Determine word boundaries in a byte buffer
- Convert an integer to its decimal text representation
- Parse command-line flags and dispatch on combinations of them
- Validate a program against an external reference rather than against expectation

---

Task

Build a line-matching tool (`rengrep`) that:

- Reads a text file and prints every line containing a literal pattern
- Supports the flags `-n`, `-c`, `-v`, `-i`, and `-w`, singly and combined
- Produces output byte-identical to `grep` for the same arguments
- Handles the awkward inputs: empty files, missing trailing newlines, patterns longer than any line, and the empty pattern

The pattern is a **fixed string**. The tool is the `grep -F` subset. The gate invokes the oracle that way. A pattern of `.` matches a literal dot and nothing wider.

Required Features

Baseline

```
./rengrep cat tests/sample.txt
```

Given `tests/sample.txt`:

```
the cat sat
The Cat Ran
concatenate
dog
cat
```

the output is:

```
the cat sat
concatenate
cat
```

Note that `concatenate` matches. The pattern is a literal substring and it appears inside the word. That's what `grep` does. The requirement is a match there.

The Flags

Flag	Behaviour	Output for the file above
(none)	Print matching lines	3 lines, as above
`-n`	Prefix each with its 1-based line number and a colon	`1:the cat sat`, `3:concatenate`, `5:cat`
`-c`	Print only how many lines matched, nothing else	`3`
`-v`	Print the lines that did **not** match	`The Cat Ran`, `dog`
`-i`	Match without regard to case	all 4 lines containing cat or Cat
`-w`	Match only where the pattern is a whole word	`the cat sat`, `cat`

`-w` excludes `concatenate`. The match there has letters on both sides. A match counts as a whole word when the character before it and the character after it are both non-word characters or absent. Word characters are letters, digits, and `_`.

Combinations

Flags combine. The combinations have to behave:

```
./rengrep -n -i cat tests/sample.txt
```

```
1:the cat sat
2:The Cat Ran
3:concatenate
5:cat
```

`-c` with `-v` counts the non-matching lines. `-c` overrides line output entirely, so `-n -c` prints just the count.

Technical Requirements

Getting the Starter

Fork cmsc-131-lab3-starter on GitHub and clone your fork. A fork is a snapshot. A fix pushed to the starter mid-week never reaches it, so the fixtures can't change under you.

Run these from Git Bash on Windows, the shell Block 1 set up, where `make` is your alias for `mingw32-make`. On Linux use your terminal.

```
git clone https://github.com/<your-account>/cmsc-131-lab3-starter.git
cd cmsc-131-lab3-starter
make
make check
```

The assembly files ship as stubs that assemble and link as-is, so the build works before any code is written. The parser stub returns an error, so `rengrep` prints its usage line and exits 2 for every command. The gate requires exit 0 for a valid invocation, so all 1345 checks fail. That total is the 1344 comparisons against `grep` and the contract pass. That red run is the correct starting state. Expect the first green cells once `args.asm` and `lines.asm` are both in place.

Write `args.asm` first, then `lines.asm`. The parser stub returns an error, so `rengrep` prints its usage line for every command until the parser works. The index stub returns no lines, so nothing prints until that works either. Both are short.

The provided files in the tree below are fixtures. The grader compares your fork against the starter, so an edited `driver.c`, `Makefile`, `run_tests.sh`, or `tests/` file shows up as a diff in the open. One member forks the repository under their account, keeps it public, and adds the rest of the group as collaborators, since that fork is the repository you submit.

Project Structure

```
rengrep/
  Makefile
  driver.c              provided: argv parsing, file reading, output
  cdecl.h               provided
  LICENSE               provided
  args.asm              yours: argv to a flag set, a pattern, and a file name
  match.asm             yours: substring search, case folding, word boundaries
```

```
  lines.asm             yours: buffer to line index
  fmt.asm               yours: integer to decimal text
  contract_test.c       provided: the second gate, in C
  contract_regs.asm     provided: register checks for contract_test
  tests/
    sample.txt          provided
    empty.txt           provided: zero bytes
    notrailing.txt      provided: no newline at end of file
    longlines.txt       provided: lines far longer than the patterns, and lines shorter
    repeated.txt        provided: many matches per line
    words.txt           provided: digits beside words and mixed case
  run_tests.sh          provided: the gate: 1344 grep comparisons and the contract test
  README.md             yours: design notes, subsystem ownership, quirks and issues
```

The starter's `README.md` ends with three sections that ship with prompts and empty tables or lists under them: Design Notes, Subsystem Ownership, and Quirks and Issues. Design Notes and Subsystem Ownership are Week 1 deliverables under the syllabus. Quirks and Issues is Week 3's. The rest of the README is the repository's own notes on the gate and the fixtures. Leave those alone.

The C Boundary

`driver.c` reads the file, loops over the lines, and writes output. It calls into your assembly for everything else:

```c
/* Builds an index of line starts and lengths. Stops at max entries and
   returns the number written. */
int PRE_CDECL index_lines(char *buf, int len, struct line *out, int max) POST_CDECL;

/* Returns 1 if pat occurs in the line, 0 otherwise.
   fold  != 0 means compare case-insensitively.
   whole != 0 means require a whole-word match. */
int PRE_CDECL line_matches(char *line, int linelen,
                           char *pat, int patlen,
                           int fold, int whole) POST_CDECL;

/* Writes the decimal text of n into out. Returns the number of bytes written. */
int PRE_CDECL int_to_dec(int n, char *out) POST_CDECL;

/* Parses argv into out. Returns 0 on success, -1 on a usage error, or the
   unknown flag character as a positive int. */
#define FLAG_N 1
#define FLAG_C 2
#define FLAG_V 4
#define FLAG_I 8
#define FLAG_W 16
struct args { int flags; char *pattern; char *path; };
int PRE_CDECL parse_args(int argc, char **argv, struct args *out) POST_CDECL;
```

`flags` is the OR of the bits. A token whose first byte is `-` and whose second byte isn't NUL is a flag group, wherever it sits on the line, so `-nvi` is `-n -v -i`. Every other token is positional. The first is the pattern, the second is the file. A third one is a usage error, and so is a missing one. `-` on its own is positional.

Don't modify `driver.c`. Your defense uses the provided copy.

The same cdecl rules from Activity 2 apply. Arguments start at `[ebp+8]`, `eax` returns the value, and `ebx`, `esi`, `edi`, and `ebp` must be preserved.

Constraints

- Use the x86 string instructions for the inner comparison. `repe cmpsb` at minimum. A byte-by-byte loop written entirely with `mov` and `cmp` doesn't meet the requirement.
- Keep the flag characters in a table and find them with `scasb`. A chain of `cmp` and `je`, one pair per flag, doesn't meet the requirement either.
- No calls into the C standard library from your assembly. No `strstr`, no `strlen`, no `tolower`.
- Don't modify the input buffer. Case folding must compare folded values without writing them back. `-i` must still print the original line as it appeared.
- Whole-number arithmetic only.

Implementation Notes

The String Instructions

`cmpsb` compares the byte at `[esi]` with the byte at `[edi]`, sets the flags, and advances both pointers. `repe` repeats while they remain equal and `ecx` isn't zero.

```
; compare patlen bytes at esi against edi
        cld                     ; clear direction flag: advance upward
        mov     ecx, patlen
        mov     esi, pattern
        mov     edi, candidate
        repe    cmpsb
        je      matched         ; ZF still set means all bytes were equal
```

`cld` matters. The direction flag decides whether the pointers advance or retreat. It persists across calls. Assuming it's clear is how a routine that works alone breaks when called after something else. Set it explicitly.

A substring search is that comparison tried at each starting position:

```
for start = 0 to linelen - patlen:
    compare patlen bytes at line+start against pattern
    if equal: return match at start
return no match
```

The bound is `linelen - patlen`, not `linelen`. Running past it compares against bytes beyond the line.

The Empty Pattern and the Over-Long Pattern

If `patlen` is greater than `linelen`, the loop must not execute at all. `linelen - patlen` is negative. Compared with an unsigned jump, a negative number reads as enormous, so the scan runs off the end of the line, through the lines after it, and into memory the process doesn't own. Then it faults.

Use signed jumps for this bound. This is the Block 5 lesson arriving where it does damage.

If `patlen` is zero, `grep` matches every line, and so must you. The bound is `linelen`, there's nothing to compare, and every position is a match. The trap is in the instruction. A `rep` prefix with `ecx` at zero executes nothing and leaves the flags as they were, so `repe cmpsb` followed by `je` tests whatever the previous instruction left in ZF. Set ZF yourself first. A `test ecx, ecx` before the `repe cmpsb` does it. ZF is 1 when the count is zero, and when the count isn't zero, `cmpsb` overwrites it anyway. A folding loop with its test at the bottom compares one byte before it checks the count, so it needs a guard of its own.

`-w ""` follows the same boundary rule as any other match. A zero-length match at offset `start` is a whole word when the byte before `start` and the byte at `start` are both non-word or absent. On `a b`, no position passes. On an empty line, position 0 passes. That's what `grep` does, and `make check` runs the empty pattern through every flag combination to hold you to it, with one exception described under Testing Strategy.

Case Folding Without Writing

`-i` must compare case-insensitively while printing the original line. So fold in a register, at comparison time.

```
; al holds a byte; fold to lowercase in place
        cmp     al, 'A'
        jb      not_upper
        cmp     al, 'Z'
        ja      not_upper
        add     al, 32              ; 'a' - 'A' = 32
not_upper:
```

This means `-i` can't use `repe cmpsb` directly, since that instruction compares raw bytes. You need a folding comparison loop for that path. Using the fast path when `fold` is zero and the folding path when it isn't is the expected structure.

Writing the folded text back into the buffer would let you use `repe cmpsb` *for both paths. Why does the specification forbid it, and what would the output look like if you did?*

Word Boundaries

A match at offset `start` of length `patlen` is a whole word when:

- `start` is 0, or the byte at `start - 1` isn't a word character, **and**
- `start + patlen` equals `linelen`, or the byte at `start + patlen` isn't a word character

Word characters are `A-Z`, `a-z`, `0-9`, and `_`.

Note that a failed word-boundary check doesn't end the search. `concatenate` fails at offset 3, but a later position in the same line might still match as a whole word. Keep scanning.

Integer to Decimal

`-n` needs line numbers as text. Divide by ten repeatedly, collecting remainders, which come out backwards:

```
; eax = number, edi = output buffer
        mov     ebx, 10
        xor     ecx, ecx            ; digit count
digit_loop:
        mov     edx, 0              ; clear before div, as always
        div     ebx                 ; eax = quotient, edx = remainder
        add     dl, '0'             ; remainder to ASCII
        push    edx                 ; stack reverses them for us
        inc     ecx
        cmp     eax, 0
        jne     digit_loop
        ; now pop ecx digits into edi in the right order
```

The stack does the reversal, and the activity requires it. Writing the digits into a temporary array and copying them backwards doesn't meet the requirement.

Zero is the case this loop gets wrong if written as a `while`. Written as above, with the test at the bottom, it emits one digit before checking, so zero prints as `0`. A top-tested loop emits nothing.

Parsing Flags with a Table

`-nvi` is three flags in one token. The driver takes them back as one integer with three bits set. Keep the flag characters in a table, in bit order, and let `scasb` find each one:

```
segment .data
flag_chars      db      'n', 'c', 'v', 'i', 'w'    ; bit 0 to bit 4

segment .text
; al = the flag character
        mov     edi, flag_chars
        mov     ecx, 5
        repne   scasb               ; scan until al matches or ecx hits 0
        jne     unknown             ; no hit: al is the unknown flag
        mov     edx, 4
        sub     edx, ecx            ; edx = the table index
```

`scasb` compares `al` with the byte at `[edi]` and advances `edi`. `repne` repeats while they differ and `ecx` isn't zero. When it stops on a hit, `ecx` has already been decremented for the matching byte too, so a hit at index `i` leaves `ecx` at `4 - i`. Read the index off that, shift 1 left by it, and `or` the bit into `flags`.

The point of the table is that adding a flag means adding a byte. A chain of `cmp al, 'n'`, `je`, `cmp al, 'c'`, `je` also works, and it grows by two instructions per flag. The rubric marks that chain down. `scasb` reads the direction flag the same way `cmpsb` does, so `cld` comes first here too.

A sixth flag isn't one change. It needs a byte in the parser's table, a single bit in the flag word, and a branch in the driver's dispatch, so the parser, the flag state, and the dispatch each move. Two of those three live in `driver.c`, which is provided and not yours to modify, so a sixth flag isn't reachable inside this activity. That's the useful answer to the architecture question: three changes in three places, and only one of them is yours.

Common Pitfalls

- Forgetting `cld`, so the string instructions run backwards depending on what ran before
- Bounding the search at `linelen` rather than `linelen - patlen`
- Using unsigned jumps on `linelen - patlen`, which runs the scan off the line and faults when the pattern is longer than the line
- A zero-length pattern reaching `repe cmpsb` with ZF still set by whatever ran before
- Folding case by modifying the buffer, so `-i` prints lowercased output
- Stopping the scan at the first word-boundary failure instead of continuing
- Off-by-one on line numbers, since `grep` counts from 1
- Reversing the `int_to_dec` digits in the output buffer, with a swap loop or a copy, instead of pushing each remainder and popping it back. The stack does the reversal. `make check` can't tell the two apart, so a passing run doesn't mean you met the requirement
- Emitting the digits most significant first, by dividing by the largest power of ten and working down. It prints the right text without the stack, so it doesn't meet the requirement either
- A file with no trailing newline losing its last line
- An empty file producing a blank line instead of no output
- Printing anything besides the number when `-c` is given
- `repe cmpsb` leaving `ecx` and the pointers moved, then reusing them without resetting
- Reading the table index from `ecx` after `repne scasb` without allowing for the decrement on the hit

- Treating - on its own as a flag group

Testing Strategy

The Oracle

Correctness is agreement with `grep`. `make check` builds the tool and the contract test, then runs `run_tests.sh`, which runs both and compares:

```
make check
```

The comparison it performs, for every flag subset and every test file:

```
LC_ALL=C grep -F $flags "$pattern" "$file" > expected.txt
./rengrep $flags "$pattern" "$file" > actual.txt
diff --strip-trailing-cr expected.txt actual.txt
```

`LC_ALL=C` keeps character classes and case folding identical on every machine, so the oracle behaves the same on your laptop and in a container. `-F` makes the pattern a fixed string, which is the subset this tool implements.

The script makes this comparison in the shell. Like the `diff` above, it drops a CR before each newline and compares every other byte. No expected-output file ships with the starter. The reference is the `grep` installed on your machine. The first run saves each `grep` output in `.grep-cache`, and later runs read the saved output. That first run starts `grep` 1344 times, so it takes the longest. The script saves the outputs again when your `grep`, `run_tests.sh`, a test file, or a saved output changes. When one of them changes during a run that reads the saved output, the script runs the tests again with `grep`. In CI it always runs `grep`. `.gitignore` keeps the cache out of your repository.

`grep` exits 0 when it matched and 1 when it didn't. The script accepts either and compares the output. That difference from your tool isn't a defect. The script requires `rengrep` to exit 0 on every valid invocation.

The script tests two `grep` behaviours. First, an empty pattern matches every line. That's what makes `-c ""` print the line count. Second, `grep -c -v ""` prints nothing at all rather than the count of 0, where every other `-c` prints a number. Add `-w`, though, and it prints the ordinary count. An empty pattern matches no whole word, so `-v` selects every line. `driver.c` reproduces both behaviours, since the tool is defined by agreement with `grep`. Measured on GNU grep 3.0 and 3.11.

Flag Coverage

Every flag subset is exercised. That means no flags, each of the five flags alone, every pair, every triple, every quadruple, and all five together. That's 32 subsets, against 7 patterns, against 6 files, or 1344 comparisons. The script runs all of them. Don't hand-test a few and assume.

The sixth pattern is `.`, which `-F` reads as a literal dot. Just one line in the corpus holds a dot, the `longlines.txt` line that spells out `run_tests.sh`, so this pattern finds that single line. That's the point. A tool that quietly implements regular expressions would match every non-empty line instead.

The Contract Pass

The oracle only sees what the driver prints. `contract_test` calls the four routines directly and checks what the output can't show:

- `index_lines` against its capacity argument, including the canary case with capacity two, where a third line must not be written

- `line_matches` on a pattern longer than the line, on the empty pattern with and without `-w`, on the word-boundary edges, and on a buffer that must come back unchanged
- `int_to_dec` on zero and on `INT_MAX`, checking both the digits and the count it reports
- `parse_args` on grouped flags, a repeated flag, a flag after the pattern, a lone dash, a missing operand, a third positional, and an unknown flag on its own and inside a group
- the register discipline of all four routines, with sentinel values in `ebx`, `esi`, and `edi` and a stack-pointer comparison

Edge Cases

Each provided file exists for a reason:

File	Tests
`empty.txt`	Zero bytes. Output must be empty, and `-c` must print `0`
`notrailing.txt`	Last line has no newline and must still be found
`longlines.txt`	Lines longer than any pattern, and lines shorter than the pattern
`repeated.txt`	Several matches on one line, which still prints the line once
`sample.txt`	The worked example above
`words.txt`	Digits beside words and mixed case, so digit word boundaries and the capitalised pattern are exercised

Also try, by hand:

- A pattern that appears at the very start of a line, and at the very end
- A pattern equal to the whole line
- A pattern one character longer than the line
- A single-character pattern
- `-w` on a line where the only match is a whole word and one where it isn't
- The empty pattern with `-w`, on a line with spaces in it and on an empty line

---

Deliverables

Your group's GitHub repository, with a clean commit history showing individual contributions, must contain:

1. `args.asm`, `match.asm`, `lines.asm`, and `fmt.asm`, documented
2. The provided `Makefile`, `driver.c`, `cdecl.h` (the build header), `run_tests.sh`, `contract_test.c`, `contract_regs.asm`, `LICENSE` (the licence), and `tests/`, unmodified, plus any test files you added
3. `README.md`, with its Design Notes, Subsystem Ownership, and Quirks and Issues sections filled in by the progress report each one is due for

Subsystem Ownership

Subsystem	Covers
Buffering and lines	The line index, its offsets and lengths, the missing trailing newline, the empty file

Subsystem	Covers
Matching	Substring search, the string instructions, case folding, word boundaries, the empty pattern
Formatting and flags	Integer to decimal, the flag table and `parse_args`, the tests you add beyond the provided six

Record the split in the Subsystem Ownership section of `README.md` before the Week 1 progress report. You'll be asked at defense about a subsystem you didn't write.

To submit, one designated member emails the group submission, sending the link to your group's public GitHub repository. CC the rest of your group. The commit you defend is the one that's graded, so push before your defense slot. Your provisional grade comes from that commit. The syllabus then gives you one week to commit any refactors your instructor requires at the defense. Those commits can raise the grade. Nothing else you push after the defense changes it. Adhere to the following subject line, joining every group member's name with `&` in the same `LastName, Initials` format: `[CMSC 131 Lab] Lab 3 Group: LastName1, Initials1 & LastName2, Initials2 & LastName3, Initials3`. A quadro adds a fourth name the same way.

Then, each member submits, individually, through email:

1. A short `reflection.txt` covering what you built on your subsystem, what fought back, and what you learned.
2. A short `peer.txt` with your honest assessment of how your other groupmates worked during the activity.

Adhere to the following subject line: `[CMSC 131 Lab] Lab 3: Surname, Initials`.

For example: `[CMSC 131 Lab] Lab 3 Group: Sanchez, SM & Reyes, JD & Cruz, AB` for the group submission and `[CMSC 131 Lab] Lab 3: Sanchez, SM` for the individual submission.

Progress Reports

The syllabus says what every progress report expects in general terms: design notes, the subsystem split, and a working prototype in Week 1, a functional prototype with the core features in Week 2, and the complete implementation with test results and documented quirks in Week 3. This section says what those mean for this activity. Each report is a 15-minute slot. Come with the repository open and `rengrep` built, and expect every member to speak.

Week 1

The specification, and a plan. This activity ends the semester, so a week lost here isn't recovered later. Bring:

- The Subsystem Ownership section of `README.md`, filled in, and a commit history that already shows who owns what
- The Design Notes section of `README.md` complete before the Week 1 progress report: the five flags and what each one does, how you plan to find the lines inside one buffer, and a timeline naming the subsystem each week finishes
- A prototype that builds and runs on `tests/sample.txt`, whatever it prints today. `args.asm` and `lines.asm` are the two stubs that unblock everything else
- Why `concatenate` matches `cat`, and what the oracle is for this activity
- Which registers you must preserve, what the direction flag controls, and where you set it

Week 2

Matching, and the first flags. Bring:

- `rengrep` with no flags on `tests/sample.txt`, agreeing with `grep` byte for byte
- Your matching loop, the bound on its scan, and what happens when the pattern is longer than the line
- `-n` agreeing with `grep`, and `-c` printing only the number
- How you fold case without changing the buffer, and your flag table handling two flags in one token
- A commit history spread across the week. Each member opens code they wrote and explains a routine inside it

Week 3

Everything green, and the edges. Bring:

- `make check` passing all 1345 checks, every flag subset against `grep` and the contract pass
- `empty.txt` printing nothing, and `-c` on it printing `0`
- The last line of `notrailing.txt` found, and each line of `repeated.txt` printed once
- The Quirks and Issues section of `README.md`, filled in, with at least one entry you can explain
- The tests you added beyond the provided six, the provided files unmodified, and three weeks of commits distributed across every member

Week 4

The defense. It takes a 25-minute slot. The window runs three days, so book early. Your instructor runs `rengrep` against `grep` and questions each member on a subsystem they didn't write. The commit you defend is the one that's graded.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, DeepSeek, etc.) to generate wholesale vibe-coded or vibe-written work is considered cheating. Cheating is against the university's code of ethics. It's subject to failure in the course, harsh disciplinary action, or expulsion. The syllabus's Academic Integrity and Honesty section is the policy authority. It applies in full.

Important Dates

This activity has no deadline of its own. It has a prescribed timeline, the four windows below, and the syllabus attaches extra credit to keeping it. The defense belongs in the Week 4 window. The one hard cutoff is course-wide: after the end of classes on December 9, no laboratory deliverable is accepted for any activity.

Progress reports and laboratory defense may be booked only during the hours defined in the syllabus, with your own laboratory instructor, on that instructor's booking page. The syllabus lists one page per instructor. Book ahead.

This is the last activity of the semester. Its defense window runs three days rather than the usual seven. It ends on the last day of classes, December 9. Nothing else is active by then, so the slots are enough to go around.

Activity	Window	Sec 3 (W)	Sec 1 & 2 (T/F)
Week 1 Progress Report	Nov 16-22	Nov 18	Nov 17, 20
Week 2 Progress Report	Nov 23-29	Nov 25	Nov 24, 27
Week 3 Progress Report	Nov 30-Dec 6	Dec 2	Dec 1, 4
Defense Window	Dec 7-9	Dec 9	Dec 8

Grading Rubric (100 Pts)

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
System Architecture (25%)	Clean split between parsing, line indexing, matching, and formatting. The folding and non-folding paths share the candidate loop and the boundary check without duplication. Flag parsing is table-driven rather than a chain of branches. `int_to_dec` reverses its digits on the stack.	Mostly modular. Some duplication between the fast and folding paths.	Matching and indexing tangled together. The boundary check is copied into both paths. Flag parsing is a long branch chain. `int_to_dec` skips the stack reversal.	No modularity. One routine does everything. The cdecl boundary is violated.
Robustness (20%)	Every flag combination matches `grep` on every test file, including the empty file, the missing trailing newline, and patterns longer than a line.	Core flags correct. A combination or an edge file disagrees in a minor way.	Common cases work but an edge file crashes or produces wrong output.	Fails the provided tests. Loops or faults on an over-long pattern.
Code Engineering (10%)	`cld` set explicitly. Callee-saved registers preserved. The input buffer never modified. Comments explain the search bound and the boundary rule.	Registers preserved. A few unexplained constants or an implicit direction flag.	Inconsistent register discipline. Buffer modified in place. Bounds uncommented.	Clobbers registers the caller needs. Corrupts the input.
Collaboration (20%)	Atomic semantic commits distributed across the month. Clear per-member ownership. Evidence of real code review between members.	Consistent version control. Adequate messages. Work visibly shared.	Inconsistent Git usage. Large dumps instead of increments. Vague messages.	Minimal history. No evidence of the whole group contributing.
Technical Defense (25%)	Every member explains the matching loop and its bound. Each traces a subsystem they didn't write. The live `grep` comparison is run and reasoned about.	Clear explanation. All members participate. The oracle comparison is understood.	Uneven participation. Struggles to explain the word-boundary rule or the search bound.	Can't explain the string instructions or the flag table. Unable to defend design decisions.